

Placement Analytics Engine

VITyarthi Java Programming Project Report

Shivansh Prashant

Registration Number: 25BAI10384

VIT Bhopal University

September 18, 2026

Contents

1	Introduction	2
2	Problem Statement	2
3	Functional Requirements	2
4	Non-functional Requirements	2
5	System Architecture	2
6	Design Diagrams	3
6.1	Use Case Diagram	3
6.2	Class Diagram	3
6.3	Sequence Diagram	4
7	Design Decisions & Rationale	4
8	Implementation Details	4
9	Screenshots / Results	5
10	Testing Approach	6
11	Challenges Faced	6
12	Learnings & Key Takeaways	6
13	Future Enhancements	6
14	References	6

1. Introduction

For this flipped course evaluation, I decided to build a Placement Analytics Engine. It is basically a CLI tool that looks at student profiles and sorts them into recruitment brackets. Normally, university placement cells do this by hand, which takes forever. This system completely automates that preliminary filtering.

2. Problem Statement

Every year, placement coordinators manually filter massive Excel sheets to see who qualifies for mass, dream, or super-dream companies. It is a huge waste of time. Plus, matching up academic CGPAs with technical coding ratings manually almost always leads to mistakes. I wanted to build something that fixes this bottleneck.

3. Functional Requirements

- **Dynamic Registration:** Users can just type in new student records directly through the terminal.
- **Automated Tier Allocation:** The logic engine assigns a placement bracket on its own using hardcoded cutoffs.
- **Analytics Reporting:** It prints out a clean, tabular summary of everyone's eligibility at the end.

4. Non-functional Requirements

- **Fault Tolerance:** If someone accidentally types a letter instead of a number, the program will not crash.
- **Memory Efficiency:** I used ArrayLists so the processing stays fast with low overhead.
- **Modular Design:** The actual business logic is completely separated from the user interface.

5. System Architecture

I went with a 5-class OOP setup. `SystemRunner` acts as the main brain. It pushes inputs to the `InputHandler` to catch any typos. From there, the `Registry` holds the `Candidate` profiles in memory, and the `EligibilityEngine` figures out which placement tier they actually belong to.

6. Design Diagrams

6.1 Use Case Diagram

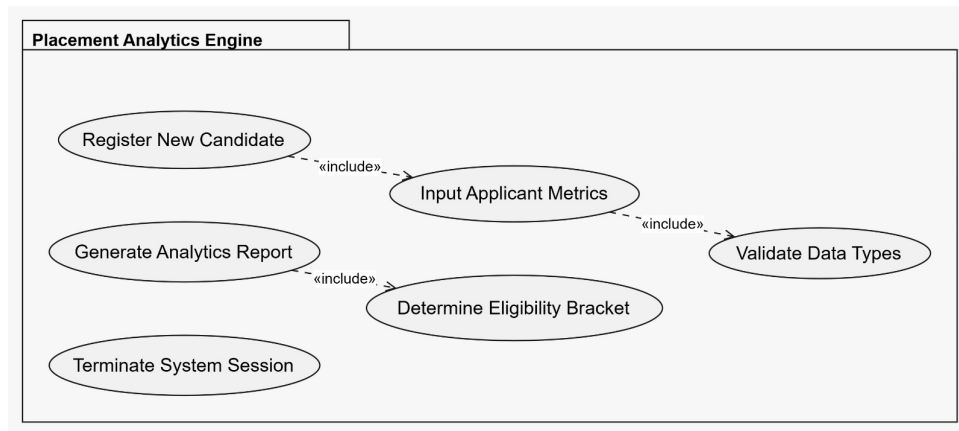


Figure 1: Use Case Diagram mapping the Placement Officer's interactions

6.2 Class Diagram

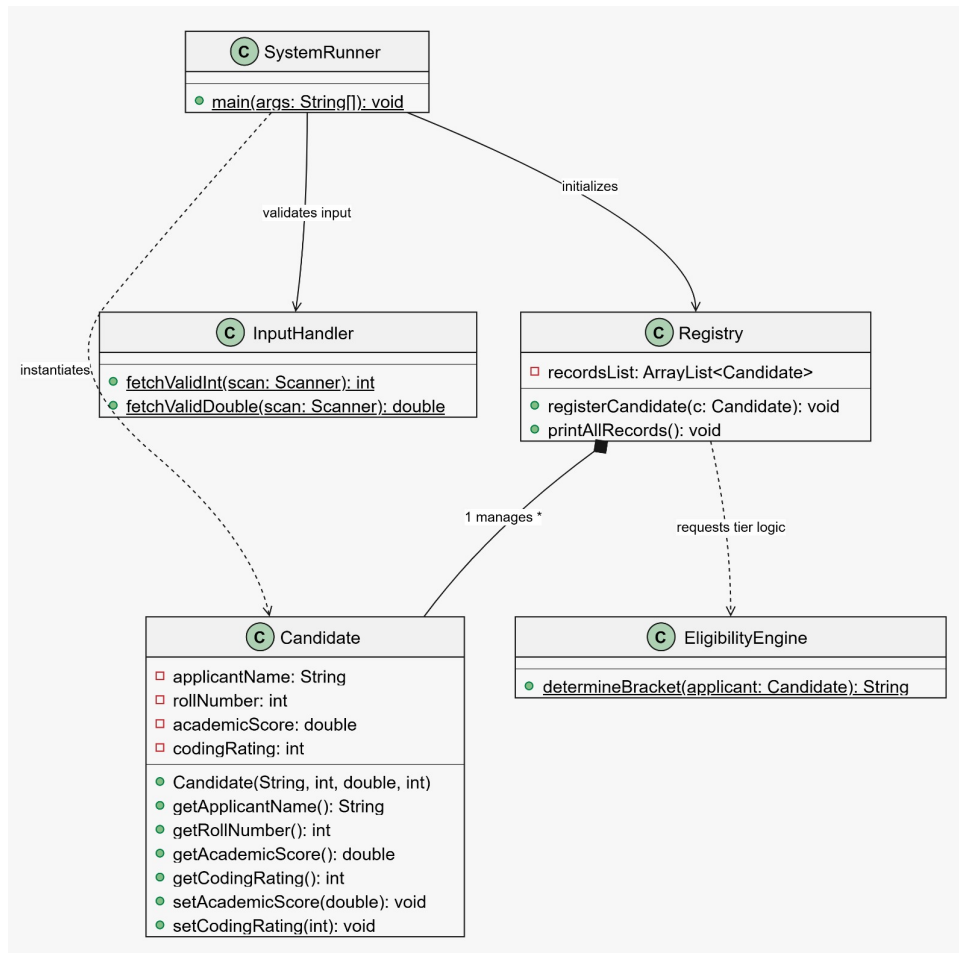


Figure 2: UML Class Diagram showing the OOP structure

6.3 Sequence Diagram

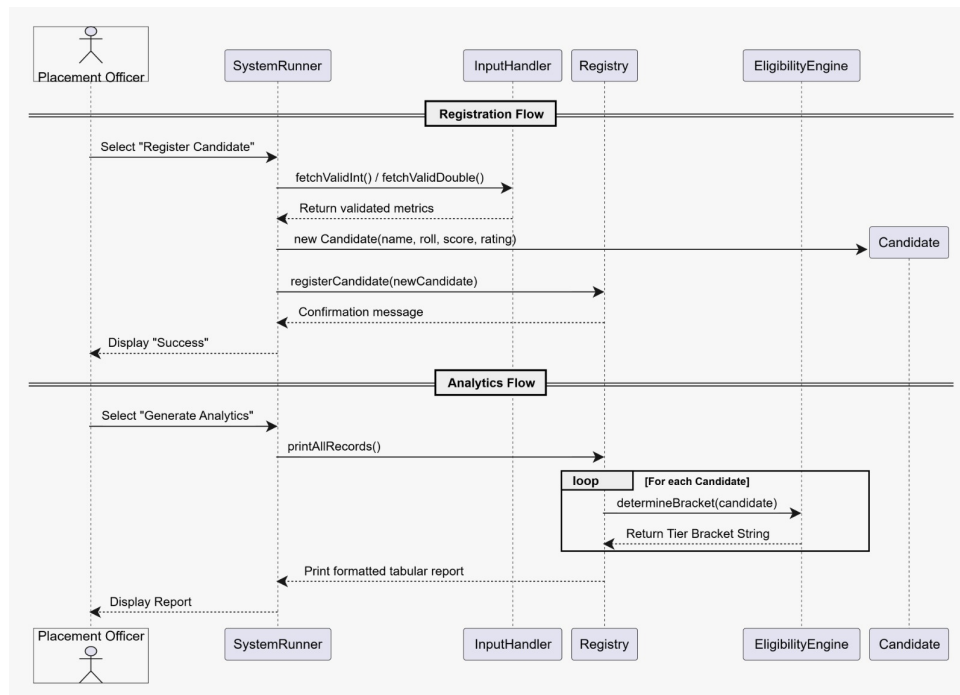


Figure 3: Sequence Diagram tracking data flow during report generation

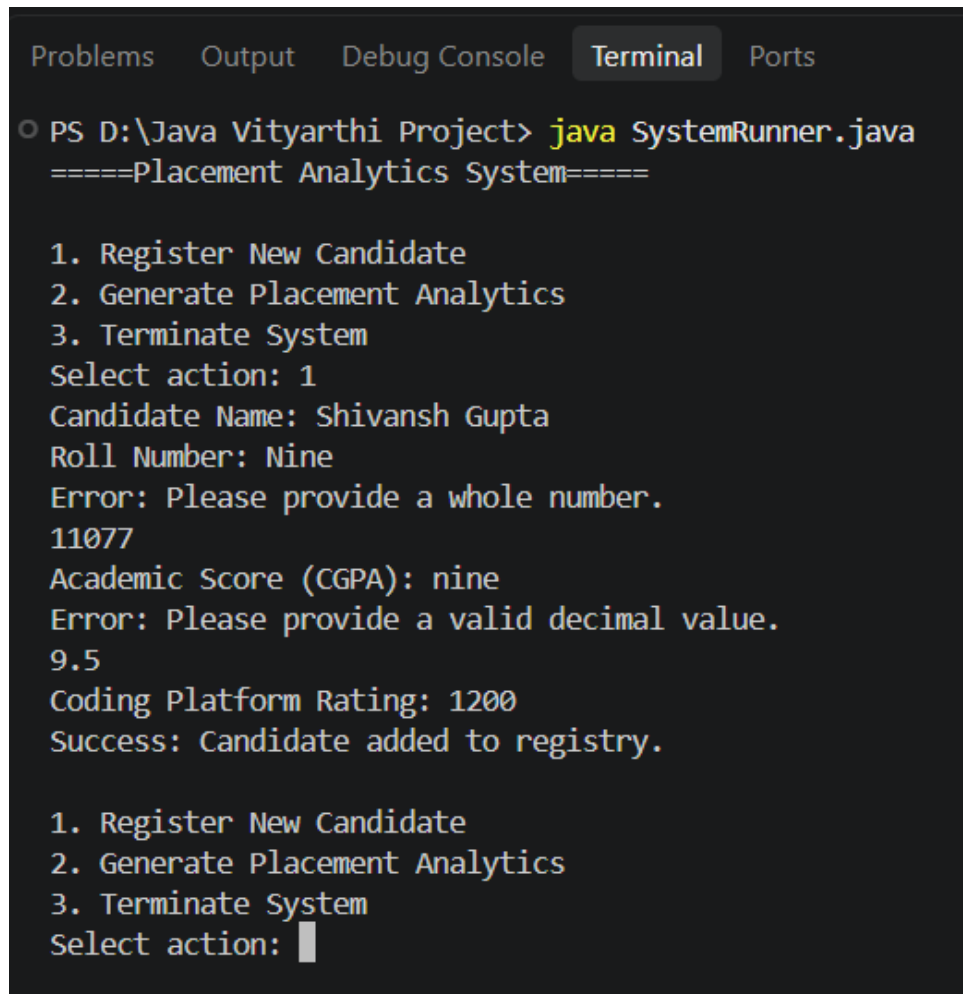
7. Design Decisions & Rationale

I skipped building a GUI because I wanted rapid execution and testing. A CLI is just way faster to work with. I chose `ArrayList` for data storage so the system could scale up dynamically, avoiding the annoying limits of hardcoded arrays. I also threw in endless `try-catch` blocks because users always mess up terminal inputs.

8. Implementation Details

The whole thing was written in Java using JDK 11. I stuck strictly to OOP rules. The `Candidate` data is fully encapsulated with private fields. I also made sure the tier-calculation logic was totally isolated in its own class rather than mixing it up with the UI code.

9. Screenshots / Results



```
Problems  Output  Debug Console  Terminal  Ports
PS D:\Java Vityarthi Project> java SystemRunner.java
=====Placement Analytics System=====

1. Register New Candidate
2. Generate Placement Analytics
3. Terminate System
Select action: 1
Candidate Name: Shivansh Gupta
Roll Number: Nine
Error: Please provide a whole number.
11077
Academic Score (CGPA): nine
Error: Please provide a valid decimal value.
9.5
Coding Platform Rating: 1200
Success: Candidate added to registry.

1. Register New Candidate
2. Generate Placement Analytics
3. Terminate System
Select action: █
```

Figure 4: Terminal output showing the system catching a bad input

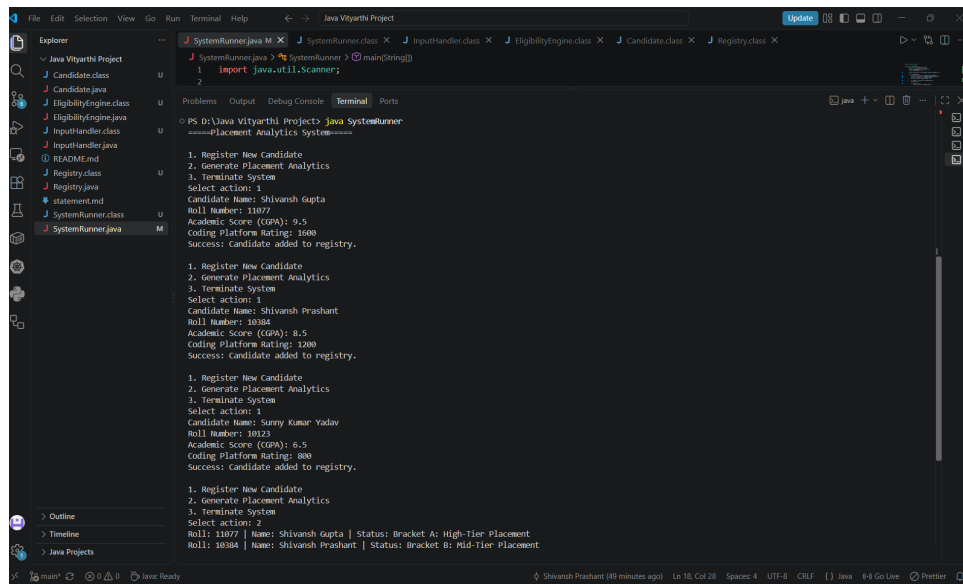


Figure 5: The final generated tabular report

10. Testing Approach

I actively tried to break my own prompts during testing. Whenever the console asked for an integer, I would intentionally type strings or random decimals. The `InputMismatchException` blocks caught these errors perfectly, cleared the scanner, and just asked me to try again without terminating the session.

11. Challenges Faced

Fighting with IDE compiler caching was a nightmare, so I actually had to compile the Java files manually using PowerShell commands. It also took some tweaking to translate real-world placement criteria into clean, nested algorithmic rules.

12. Learnings & Key Takeaways

I finally got to use exception handling in a real scenario instead of just reading theory. PlantUML turned out to be super useful for mapping architecture, and I realized how easily bad Git commands can mess up a repository.

13. Future Enhancements

When we close the terminal the data gets deleted as of now but later i would connect this with a database probably like MySQL. Can also build a GUI for this.

14. References

- Oracle Java SE Documentation (Exception Handling)
- PlantUML Language Reference Guide